

smart-build 独立构建系统技术实施方案

项目目标

将 smart-build 项目从 yocto/poky 依赖中解耦，使其成为一个独立的构建系统，仅依赖于 bitbake，实现完全自主的 RT-Smart 构建环境。

技术架构设计

目标架构

```
smart-build/
├── bitbake/                # BitBake 构建引擎 (git submodule)
├── meta-core/              # 核心基础层 (替代 poky/meta)
│   ├── conf/
│   │   ├── layer.conf
│   │   ├── bitbake.conf
│   │   └── machine/
│   │       ├── qemuarm64.conf
│   │       └── qemuriscv64.conf
│   └── distro/
│       └── smart.conf
├── classes/
└── recipes-core/
├── meta-rt-smart/          # RT-Smart 专用层 (现有)
├── smart-env               # 环境初始化脚本
├── conf/
│   └── bblayers.conf.sample # 层配置模板
└── build/                  # 构建目录 (动态生成)
    └── conf/
        ├── local.conf
        └── bblayers.conf
```

分阶段实施步骤

阶段一：环境准备和依赖分析 (1-2天)

步骤 1.1：分析 poky/meta 核心依赖

目标：识别 meta-rt-smart 实际依赖的核心组件

实施内容：

- 分析现有 meta-rt-smart 中的配方依赖关系
- 识别 poky/meta 中的必需组件：
 - 基础类文件 (classes/*.bbclass)
 - 核心配置文件 (conf/bitbake.conf)
 - 机器配置文件 (conf/machine/)
 - 发行版配置文件 (conf/distro/)
- 创建依赖关系图和最小化组件清单

验收标准:

- ☐ 完成依赖分析报告
- ☐ 确定最小化核心组件清单
- ☐ 制定组件迁移计划

步骤 1.2: 添加 BitBake 子模块

目标: 将 BitBake 作为独立组件集成到项目中

实施内容:

```
# 在 smart-build 根目录执行
git submodule add https://github.com/openembedded/bitbake.git
git submodule update --init --recursive
```

验收标准:

- ☐ BitBake 子模块成功添加
- ☐ 能够独立运行 BitBake 命令
- ☐ 版本锁定到稳定分支

阶段二: 创建核心基础层 meta-core (2-3天)

步骤 2.1: 创建 meta-core 层结构

目标: 建立独立的核心基础层

实施内容:

```
mkdir -p meta-core/{conf/{machine,distro},classes,recipes-core}
```

关键文件创建:

1. meta-core/conf/layer.conf

```
# Layer配置
BBPATH .= ":${LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/**/*.bb ${LAYERDIR}/recipes-*/**/*.bbappend"
BBFILE_COLLECTIONS += "core"
BBFILE_PATTERN_core = "^${LAYERDIR}/"
BBFILE_PRIORITY_core = "5"
LAYERSERIES_COMPAT_core = "styhead"
```

2. meta-core/conf/bitbake.conf

```
# 基础构建配置
BUILD_ARCH = "${@os.uname()[4]}"
BUILD_OS = "${@os.uname()[0].lower()}"
BUILD_VENDOR = ""
BUILD_SYS = "${BUILD_ARCH}${BUILD_VENDOR}-${BUILD_OS}"

# 工作目录配置
TMPDIR ?= "${TOPDIR}/tmp"
WORKDIR = "${TMPDIR}/work/${MULTIMACH_TARGET_SYS}/${PN}/${EXTENDPE}${PV}-${PR}"
```

```

S = "${WORKDIR}/${BP}"
B = "${S}"

# 基础工具链设置
TARGET_ARCH ?= "${MACHINE_ARCH}"
TARGET_OS = "linux"
TARGET_VENDOR = ""
TARGET_SYS = "${TARGET_ARCH}${TARGET_VENDOR}-${TARGET_OS}"

# 包管理
PACKAGE_CLASSES ?= "package_rpm"

```

验收标准:

- ☐ meta-core 层结构创建完成
- ☐ 基础配置文件就位
- ☐ 层可以被 BitBake 识别

步骤 2.2: 创建机器配置文件

目标: 定义支持的目标平台

实施内容:

1. meta-core/conf/machine/qemuarm64.conf

```

#@TYPE: Machine
#@NAME: QEMU ARMv8 machine
#@DESCRIPTION: Machine configuration for running an ARMv8 system on QEMU

require conf/machine/include/arm/arch-armv8a.inc

KERNEL_IMAGETYPE = "Image"
SERIAL_CONSOLES ?= "115200;ttyAMA0 115200;hvc0"

MACHINE_FEATURES = "kernel26 apm usb gadget usbhost vfat alsa"

# Toolchain settings
TOOLCHAIN_HOST_TASK ?= ""
TOOLCHAIN_TARGET_TASK ?= ""

# RT-Smart specific settings
RTSMART_BSP = "qemu-virt64-aarch64"
RTSMART_ARCH = "aarch64"
RTSMART_TOOLCHAIN = "aarch64-linux-musleabi"

```

2. meta-core/conf/machine/qemuriscv64.conf

```

#@TYPE: Machine
#@NAME: QEMU RISC-V 64-bit machine
#@DESCRIPTION: Machine configuration for running a RISC-V 64-bit system on QEMU

DEFAULTTUNE ?= "riscv64"
require conf/machine/include/riscv/tune-riscv.inc

KERNEL_IMAGETYPE = "Image"

```

```
SERIAL_CONSOLES ?= "115200;ttys0"

MACHINE_FEATURES = "kernel26 usb gadget usbhost vfat"

# RT-Smart specific settings
RTSMART_BSP = "qemu-virt64-riscv"
RTSMART_ARCH = "riscv64"
RTSMART_TOOLCHAIN = "riscv64-linux-musleabi"
```

验收标准:

- ☐ 机器配置文件创建完成
- ☐ 配置参数正确设置
- ☐ BitBake 能够解析机器配置

步骤 2.3: 创建发行版配置

目标: 定义 smart 发行版特性

实施内容:

meta-core/conf/distro/smart.conf

```
#@TYPE: Distribution
#@NAME: Smart
#@DESCRIPTION: RT-Smart Distribution

DISTRO = "smart"
DISTRO_NAME = "RT-Smart"
DISTRO_VERSION = "1.0"
DISTRO_CODENAME = "smart"

MAINTAINER = "RT-Thread Team <support@rt-thread.org>"

# Feature settings
DISTRO_FEATURES = "largefile multiarch"
DISTRO_FEATURES_BACKFILL_CONSIDERED = "pulseaudio sysvinit"

# Toolchain settings
TCMODE ?= "default"
GCCVERSION ?= "11.%"

# Package management
PACKAGE_CLASSES ?= "package_tar"

# Init system
INIT_MANAGER = "none"

# Security flags
SECURITY_CFLAGS = ""
SECURITY_LDFLAGS = ""
```

验收标准:

- ☐ 发行版配置文件创建完成
- ☐ 特性设置符合 RT-Smart 需求

☐ BitBake 能够解析发行版配置

阶段三：创建环境初始化脚本 (1天)

步骤 3.1：创建 smart-env 脚本

目标：替代 poky 的 oe-init-build-env 脚本

实施内容：

smart-env 脚本

```
#!/bin/bash

# smart-env - RT-Smart Build Environment Setup Script
# Usage: source smart-env [build_dir]

if [ -n "$BASH_SOURCE" ]; then
    THIS_SCRIPT=$BASH_SOURCE
elif [ -n "$ZSH_NAME" ]; then
    THIS_SCRIPT=$0
else
    THIS_SCRIPT="$(pwd)/smart-env"
fi

if [ -n "$BBSERVER" ]; then
    unset BBSERVER
fi

SMART_BUILD_ROOT=$(cd $(dirname $THIS_SCRIPT) && pwd)
BUILD_DIR=${1:-build}

# 创建构建目录
if [ ! -d "$BUILD_DIR" ]; then
    mkdir -p "$BUILD_DIR/conf"
    echo "Created build directory: $BUILD_DIR"
fi

# 设置环境变量
export SMART_BUILD_ROOT
export BBPATH="$SMART_BUILD_ROOT/meta-core:$SMART_BUILD_ROOT/meta-rt-smart"
export BB_ENV_EXTRAWHITE="SMART_BUILD_ROOT MACHINE DISTRO"
export PATH="$SMART_BUILD_ROOT/bitbake/bin:$PATH"

# 创建配置文件
cd "$BUILD_DIR"

# 创建 local.conf
if [ ! -f conf/local.conf ]; then
    cat > conf/local.conf << EOF
# RT-Smart Build Configuration
#
# Machine Selection
MACHINE ??= "qemuarm64"

# Distribution Selection
DISTRO ?= "smart"
    EOF
fi
```

```

# Build directories
TMPDIR = "\${TOPDIR}/tmp"
DL_DIR ?= "\${TOPDIR}/downloads"
SSTATE_DIR ?= "\${TOPDIR}/sstate-cache"

# Parallelism Options
BB_NUMBER_THREADS ?= "\${@oe.utils.cpu_count()}"
PARALLEL_MAKE ?= "-j \${@oe.utils.cpu_count()}"

# Package Management
PACKAGE_CLASSES ?= "package_tar"

# Additional settings
USER_CLASSES ?= "buildstats"
PATCHRESOLVE = "noop"

# QA checks
WARN_QA = "textrel files-invalid incompatible-license xorg-driver-abi
buildpaths"
ERROR_QA = "dev-so debug-deps dev-deps debug-files arch pkgconfig la perms
useless-rpaths rpaths staticdev ldflags pkgvarcheck already-stripped compile-
host-path install-host-path pn-overrides infodir build-deps file-rdeps version-
going-backwards host-user-contaminated uppercase-pn patch-fuzz-warning"
EOF
    echo "Created conf/local.conf"
fi

# 创建 bblayers.conf
if [ ! -f conf/bblayers.conf ]; then
    cat > conf/bblayers.conf << EOF
# Layer Configuration
POKY_BBLAYERS_CONF_VERSION = "2"

BBPATH = "\${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \\\
    ${SMART_BUILD_ROOT}/meta-core \\\
    ${SMART_BUILD_ROOT}/meta-rt-smart \\\
"
EOF
    echo "Created conf/bblayers.conf"
fi

echo ""
echo "### RT-Smart Build Environment Ready ###"
echo ""
echo "Build Configuration:"
echo "  Build Directory: $(pwd)"
echo "  Machine: \${MACHINE:-qemuarm64}"
echo "  Distribution: \${DISTRO:-smart}"
echo ""
echo "Common build targets:"
echo "  bitbake smart-gcc -c install_toolchain"
echo "  bitbake busybox -c build_rootfs"
echo "  bitbake rt-smart -c build_kernel"
echo "  bitbake rt-smart -c build_all"
echo ""

```

验收标准:

- ☐ smart-env 脚本创建完成
- ☐ 能够正确设置构建环境
- ☐ 自动创建必要的配置文件
- ☐ 环境变量设置正确

阶段四：适配现有配方 (1-2天)

步骤 4.1：更新 meta-rt-smart 配方

目标：确保现有配方与新的独立环境兼容

实施内容:

1. 检查并更新 layer.conf

```
# 更新 meta-rt-smart/conf/layer.conf
LAYERDEPENDS_meta-rt-smart = "core" # 改为依赖 meta-core
```

2. 验证配方兼容性

- 检查所有 .bb 文件中的变量引用
- 确认工具链路径配置正确
- 验证构建依赖关系

验收标准:

- ☐ 所有配方通过语法检查
- ☐ 依赖关系正确配置
- ☐ 与新环境完全兼容

步骤 4.2：创建必要的基础类文件

目标：提供配方执行所需的基础类

实施内容:

1. meta-core/classes/base.bbclass

```
# 基础构建类
BB_DEFAULT_TASK ?= "build"

addtask build
do_build[noexec] = "1"
do_build[recrdeptask] = "do_populate_sysroot do_deploy"

python base_do_unpack() {
    src_uri = (d.getVar('SRC_URI') or "").split()
    if not src_uri:
        return

    fetcher = bb.fetch2.Fetch(src_uri, d)
    fetcher.unpack(d.getVar('WORKDIR'))
}
```

```
addtask unpack after do_fetch
do_unpack[dirs] = "${WORKDIR}"
```

验收标准:

- ☐ 基础类文件创建完成
- ☐ 提供必要的构建任务
- ☐ 配方能够正常继承使用

阶段五：集成测试和验证 (1-2天)

步骤 5.1：功能测试

目标：验证独立构建系统的完整功能

测试用例:

```
# 1. 环境初始化测试
source smart-env
echo "Environment: PASS/FAIL"

# 2. 配置解析测试
bitbake -e | grep MACHINE
echo "Configuration parsing: PASS/FAIL"

# 3. 工具链安装测试
bitbake smart-gcc -c install_toolchain
echo "Toolchain installation: PASS/FAIL"

# 4. 文件系统构建测试
bitbake busybox -c build_rootfs
echo "Rootfs build: PASS/FAIL"

# 5. 内核编译测试
bitbake rt-smart -c build_kernel
echo "Kernel build: PASS/FAIL"

# 6. 完整构建测试
bitbake rt-smart -c build_all
echo "Full build: PASS/FAIL"

# 7. QEMU 启动测试
cd build/qemuarm64
./run_qemuarm64.sh &
sleep 5
kill %1
echo "QEMU launch: PASS/FAIL"
```

验收标准:

- ☐ 所有测试用例通过
- ☐ 构建产物正确生成
- ☐ QEMU 能够正常启动系统

步骤 5.2：多架构验证

目标：确保两种架构都能正常工作

测试内容：

```
# ARM64 架构测试
echo 'MACHINE = "qemuarm64"' >> build/conf/local.conf
bitbake rt-smart -c build_all

# RISC-V64 架构测试
sed -i 's/qemuarm64/qemuriscv64/' build/conf/local.conf
bitbake rt-smart -c cleanall
bitbake rt-smart -c build_all
```

验收标准：

- ☐ ARM64 架构构建成功
- ☐ RISC-V64 架构构建成功
- ☐ 两种架构都能正常运行

阶段六：文档和优化 (1天)

步骤 6.1：更新项目文档

目标：提供完整的使用说明

实施内容：

- 更新 README.md
- 创建架构设计文档
- 编写故障排除指南
- 提供迁移指南

步骤 6.2：性能优化

目标：提升构建效率

优化项目：

- 并行构建配置优化
- 缓存策略配置
- 下载源优化配置

风险评估和应对策略

主要风险

- 依赖复杂性风险**
 - 风险：可能漏掉关键的 poky 依赖组件
 - 应对：分阶段测试，逐步替换验证
- 兼容性风险**
 - 风险：新环境与现有配方不兼容
 - 应对：保持向后兼容，渐进式迁移
- 维护成本风险**
 - 风险：需要维护独立的核心层
 - 应对：最小化核心组件，关注必需功能

回滚策略

如果出现重大问题，可以快速回滚到原有的 poky 依赖方案：

- 1. 保留原有的使用文档
- 2. 通过分支管理保持两套方案并存
- 3. 提供自动化的环境切换脚本

预期收益

- 1. **独立性**：完全摆脱对 yocto/poky 的依赖
- 2. **轻量化**：减少不必要的组件和依赖
- 3. **可控性**：完全掌控构建环境和流程
- 4. **扩展性**：更容易添加新的架构和功能
- 5. **维护性**：简化的架构便于长期维护

实施时间表

阶段	时间	主要任务	交付物
阶段一	1-2天	环境准备和依赖分析	依赖分析报告、BitBake 集成
阶段二	2-3天	创建 meta-core 层	完整的核心基础层
阶段三	1天	创建环境脚本	smart-env 脚本
阶段四	1-2天	适配现有配方	兼容的配方文件
阶段五	1-2天	集成测试验证	测试报告和验证结果
阶段六	1天	文档和优化	完整文档和优化配置

总预期时间：7-11 个工作日

成功标准

- 1. ☒ 能够在不依赖 poky 的情况下完成完整构建
- 2. ☒ 支持 qemuarm64 和 qemuriscv64 两种架构
- 3. ☒ 构建产物与原方案完全一致
- 4. ☒ 构建时间不超过原方案的 120%
- 5. ☒ 提供完整的使用文档和示例
- 6. ☒ 通过所有现有的功能测试用例